第 7 章：函数

Python 函数的核心不只是 `def`，而是两件事：**调用时如何把实参绑定给形参**，以及**函数本身也能像普通对象一样被传递。**

这两点解释了默认参数、关键字参数、`*args`、`**kwargs`、回调、排序键和闭包。对 Java 工程师来说，可以先把 Python 函数理解成“自带一套灵活绑定规则的可调用对象”，但不能把它完全等同于 Java 方法：Python 默认没有编译期重载选择，类型提示也不会自动做运行时校验。

配套代码：`example.py / Chapter07Functions.java`。

学习目标

学完本章后，你应该能够：

- 解释执行 `def` 与调用函数分别发生了什么；
- 判断位置参数、关键字参数、默认值、`/` 和 `*` 如何共同决定参数绑定；
- 避免可变默认值共享、重复传参和不受约束的 `**kwargs`；
- 区分函数对象与函数调用结果，并把函数作为参数或返回值；
- 理解闭包捕获、`lambda` 的用途与边界；
- 同时读懂定义侧和调用侧的 `*`、`**`；
- 用常见内置函数替代重复的计数、排序、配对和聚合代码。

先建立 Java → Python 心智模型

要解决的问题	Java 中的常见方式	Python 中的常见方式	不能直接等同的地方
定义可复用行为	类中的方法、 静态方法	模块级函数、方法、局部函数	函数是运行时对象，可以赋值、 传参和返回
提供缺省配置	方法重载、Builder	默认参数、关键字参数	默认值在执行 `def` 时求值， 不是每次调用都求值
限制调用形式	方法签名	`/`、`*` 和明确的参数名	参数名可能成为公共 API 的一部分
			函数内部得到元组；调用侧 `*`

接收任意数量的值	Java varargs	`*args`	函数内部得到元组；调用侧 `*` 还能拆开任意可迭代对象
接收命名配置	配置对象、 `Map`	明确的仅关键字参数或 `**kwargs`	`**kwargs` 灵活，但会削弱自动补全和拼写检查
传递一段行为	函数式接口、lambda、method reference	直接传函数或其他可调用对象	不要求先声明专用函数式接口，类型提示也不强制校验
返回多个结果	DTO、数组	常用元组打包与解包	`return a, b` 实际只返回一个二元组对象

7.1 从 `def` 到 `return`：函数调用发生了什么

`def` 创建函数，括号触发调用

`def` 是一条可执行语句。程序执行到它时，Rython 创建一个函数对象，并把函数名绑定到该对象；函数体要等到调用时才执行：

```
def add(left: int, right: int) -> int:
    result = left + right
    return result


operation = add       # 保存函数对象，没有执行函数体
answer = operation(10, 20)  # 调用函数，得到 30
```

`left` 、 `right` 是定义中的**形参**（parameter）， `10` 、 `20` 是调用时提供的**实参**（argument）。一次普通调用可以用下面的简化过程理解：

1. 先计算调用表达式中的各个实参；
2. 按函数签名把实参绑定给形参；
3. 如果缺少参数、参数重复或出现不允许的参数，抛出 `TypeError` ，函数体不会开始执行；
4. 绑定成功后，使用这次调用的局部名字执行函数体；
5. 遇到 `return` 时立即结束本次调用，把结果交给调用者。

没有显式 `return` ，或者只写 `return` ，都会返回 `None` ：

```python
def print_message(message: str) -> None:
    print(message)


result = print_message("hello")
print(result)  # None
```

`return` 后写逗号分隔的多个值时，Python 会先把它们组成一个元组：

```python
def divide(dividend: int, divisor: int) -> tuple[int, int]:
    return dividend // divisor, dividend % divisor


quotient, remainder = divide(17, 5)  # 实际返回 (3, 2)，随后被解包
```

类型提示不是运行时门卫

`left: int` 和 `-> int` 是类型提示。它们服务于读者、IDE、Pyright 和 mypy，但 Python 默认不会像 Java 编译器那样阻止错误类型进入函数：

```python
def double(number: int) -> int:
    return number * 2


double("ha")  # 运行成功，结果是 "haha"，尽管它不符合类型提示
```

Python 是动态类型语言，不是“没有类型”：`"ha"` 仍然是运行时类型为 `str` 的对象，只是形参不会像 Java 变量那样在编译期限制它能绑定的对象。业务入口需要强校验时，要显式校验，或者交给数据校验框架；类型提示本身不负责。

参数传递的是对象，重新绑定不等于修改对象

可以把 Python 的参数传递先类比为 Java 的“对象引用按值传递”，但 Python 中整数、函数、类等也都是对象。调用后，形参只是新绑定到调用者提供的对象：

```python
def add_item(items: list[str]) -> None:
```

7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代_ 商家下单

参数传递的是对象，重新绑定不等于修改对象

可以把 Python 的参数传递先类比为 Java 的"对象引用按值传递"，但 Python 中整数、函数、类等也都是对象。调用后，形参只是新绑定到调用者提供的对象：

```
def add_item(items: list[str]) -> None:
    items.append("Python")  # 修改双方共享的列表对象


def replace_items(items: list[str]) -> None:
    items = ["new"]         # 只把局部名字绑定到新列表

courses = ["Java"]
add_item(courses)       # courses 变为 ["Java", "Python"]
replace_items(courses)  # courses 不变
```

判断调用是否会影响外部时，不要只看"有没有赋值"，而要看函数是在**修改共享对象**，还是在**重新绑定局部名字**。

同名 `def` 不是方法重载

Python 不会根据参数数量或类型，在多个同名定义之间选择版本：

```
def greet(name: str) -> str:
    return f"你好，{name}"


def greet(name: str, greeting: str) -> str:
    return f"{greeting}，{name}"


greet("Alice")  # TypeError：名字 greet 已经指向第二个函数
```

第二次执行 `def` 是把名字 `greet` 重新绑定到新函数对象。Python 通常用默认参数和关键字参数表达多种调用形式；语义不同的操作应使用不同函数名。确实需要按运行时类型分派时，可以了解 `functools.singledispatch`，但它也不是 Java 的通用方法重载。

7.2 位置参数与默认参数：先理解绑定，再写调用

README
vf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单

7.2 位置参数与默认参数：先理解绑定，再写调用

位置参数按顺序绑定

```
def introduce(name: str, city: str) -> str:
    return f"我是 {name}，来自 {city}"


introduce("Alice", "上海")
```

第一个实参绑定给 `name`，第二个绑定给 `city`。如果两个参数类型相同，顺序传反通常不会触发技术错误，却会产生业务错误。这也是参数较多时应考虑关键字调用的原因。

参数数量不匹配会在进入函数体之前失败：

```
introduce("Alice")              # TypeError: 缺少 city
introduce("Alice", "上海", 28)  # TypeError: 多了一个位置实参
```

默认值只在调用者省略参数时使用

```
def greet(name: str, greeting: str = "你好") -> str:
    return f"{greeting}，{name}"


greet("Alice")              # "你好，Alice"
greet("Alice", "早上好")    # "早上好，Alice"
greet("Alice", greeting="晚上好")
```

在普通参数区间内，没有默认值的必填参数必须位于有默认值的参数之前：

```
def valid(required, optional=0): ...
# def invalid(optional=0, required): ...  # SyntaxError
```

这份签名可以覆盖 Java 中常见的两个重载，但它还有一条非常重要的运行时规则。

可变默认值为什么会跨调用共享

可变默认值为什么会跨调用共享

默认值在执行 `def` 时计算一次，并保存在函数对象上，不会在每次调用时重新计算。因此，把列表、字典或集合直接作为默认值，会让多次调用共享同一个对象：

```
def unsafe_collect(item: str, target: list[str] = []) -> list[str]:
    target.append(item)
    return target


first = unsafe_collect("A")
second = unsafe_collect("B")
print(first)            # ['A', 'B']
print(first is second)  # True
```

安全写法是用 `None` 表示“调用者没有提供”，再在函数体内创建对象：

```
def collect(item: str, target: list[str] | None = None) -> list[str]:
    actual_target = [] if target is None else target
    actual_target.append(item)
    return actual_target
```

这份代码有两种明确语义：

- 省略 `target`：每次调用创建新列表，调用之间互不影响；
- 显式传入列表：函数修改调用者的列表，并返回同一个对象。

如果函数承诺不修改调用者的数据，应在收到列表时复制，例如 `actual_target = list(target)`。如果 `None` 本身也是一个合法业务值，可以创建专用哨兵对象来区分“未提供”和“提供了 `None`”。

同一规则也适用于 `datetime.now()`、随机数和普通函数调用：把它们写成默认值，会在定义函数时执行。需要“本次调用的当前时间”时，应在函数体内计算。

7.3 关键字参数、`/` 与 `*`：主动设计调用边界

关键字参数把业务含义写在调用处

README
f7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html
关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

关键字参数把业务含义写在调用处

比较两种调用：

```
connect("db.example.com", 5432, False, 10.0)
connect("db.example.com", port=5432, use_ssl=False, timeout=10.0)
```

第二种写法让布尔值和数字的含义一目了然。布尔开关、多个同类型参数和非必填配置尤其适合关键字调用。

同一个形参不能同时按位置和名称获得两个值：

```
connect("db.example.com", host="localhost")
# TypeError: host 同时收到了位置实参和关键字实参
```

调用者一旦写下 `timeout=10.0`， `timeout` 就成为公共 API 契约的一部分。以后把它改名为 `timeout_seconds`，会破坏已有调用；公共函数的参数名需要像方法名一样认真设计。

`/` 和 `*` 把参数分成三个区域

观察这个签名：

```
def price(amount, /, tax_rate=0.13, *, discount=0.0):
    discounted = amount * (1 - discount)
    return round(discounted * (1 + tax_rate), 2)
```

`/` 和裸 `*` 只负责划分边界，它们不是形参：

参数所在位置	参数种类	合法传法
`/` 之前的 `amount`	仅位置参数	只能写 `price(100.0)`
`/` 与 `*` 之间的 `tax_rate`	位置或关键字参数	`price(100.0, 0.1)` 或 `tax_rate=0.1`
`*` 之后的 `discount`	仅关键字参数	必须写 `discount=0.2`

```
price(100.0)
price(100.0, 0.10, discount=0.20)
price(100.0, tax_rate=0.10, discount=0.20)
```

README
t72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计费

```
price(100.0)
price(100.0, 0.10, discount=0.20)
price(100.0, tax_rate=0.10, discount=0.20)

price(amount=100.0)        # TypeError: amount 只能按位置传递
price(100.0, 0.10, 0.20)  # TypeError: discount 只能按名称传递
```

设计接口时可以这样判断：

- 参数像 `len(obj)` 中的 `obj` 一样自然，而且参数名不应成为契约时，可以使用仅位置参数；
- 参数名能解释值的业务含义，特别是开关和配置项时，优先使用仅关键字参数；
- 简短的内部函数不必为了展示语法而强行加入 `/`；
- 稳定公共接口可以主动限制调用方式，让误用尽早变成 `TypeError`。

用一张表看完整绑定模型

Python 的参数按以下顺序出现在定义中：

```
def inspect_call(
    identifier,          # 仅位置参数
    /,
    category="general", # 位置或关键字参数
    *tags,               # 收集剩余位置实参
    active=True,         # 仅关键字参数
    **extra,             # 收集剩余关键字实参
):
    return identifier, category, tags, active, extra
```

调用：

```
result = inspect_call(
    1001,
    "course",
    "Python",
    "beginner",
    active=True,
    owner="Alice",
)
```

绑定结果是：

绑定结果是：

形参	得到的值	原因
`identifier`	`1001`	第一个位置实参，只能按位置绑定
`category`	`"course"`	下一个位置实参覆盖默认值
`tags`	`("Python", "beginner")`	剩余位置实参被收集为元组
`active`	`True`	通过名称绑定仅关键字参数
`extra`	`{"owner": "Alice"}`	未被明确形参接收的关键字实参进入字典

只要记住“先按签名找到归属，再补默认值，最后检查缺失、重复和多余参数”，复杂签名就不必靠猜。

7.4 常用内置函数：优先复用 Python 的数据协议

许多 Python 内置函数面向协议工作，而不是绑定到某个具体类。例如，只要对象能报告长度，`len(obj)` 就能使用；字符串、列表、元组、集合和字典都支持它。

内置函数	作用	关键边界
`len(items)`	返回元素数量	字典计算键值对数量，字符串计算字符数量
`sum(numbers)`	数值求和	空输入结果为 `0`；不要用来拼字符串
`min` / `max`	找最小值或最大值	空输入抛出 `ValueError`；可用 `key` 提取比较依据
`sorted(items)`	返回排序后的新列表	不修改原数据；`list.sort()` 才原地修改列表
`enumerate(items)`	同时产生序号和值	`start=1` 可生成面向用户的编号
`zip(a, b)`	按位置配对多个可迭代对象	默认到最短输入为止；等长是约束时用 `strict=True`
`any(items)`	是否至少一个元素为真	`any([])` 是 `False`
`all(items)`	是否所有元素都为真	`all([])` 是 `True`，业务上可能需要额外处理空输入

```
scores = [88, 95, 72, 100]

sum(scores)                              # 355
sorted(scores, reverse=True)             # [100, 95, 88, 72]
any(score == 100 for score in scores)    # True
all(score >= 60 for score in scores)     # True
```

`key` 是“提取比较键”，不是两两比较器

```
words = ["pear", "fig", "banana"]
sorted(words, key=len)  # ['fig', 'pear', 'banana']
```

`sorted` 会把每个元素交给 `key` 函数，得到用于排序的值。这里比较的是 `3、4、6`，而不是让 `len` 比较两个单词。这和 Java `Comparator` 的二元比较模型不同，更接近先执行一次 `map` 提取排序键，再按键排序。

需要多个排序条件时，可以返回元组：

```
sorted(words, key=lambda word: (len(word), word))
```

Python 按元组元素从左到右比较，所以先按长度，再按单词本身排序。

`enumerate` 和严格 `zip` 让结构错误更早暴露

```
for number, name in enumerate(["Alice", "Bob"], start=1):
    print(number, name)

names = ["Alice", "Bob"]
scores = [95, 88]
pairs = list(zip(names, scores, strict=True))
```

普通 `zip` 在最短输入耗尽时停止，可能静默丢掉较长一侧的数据。两边本应严格等长时，`strict=True` 会在长度不一致时抛出 `ValueError`。

7.5 函数也是对象：把行为放进数据流

函数是一等对象”意味着函数可以绑定给变量、放进容器、作为实参传入，也可以从另一个函数

7.5 函数也是对象：把行为放进数据流

“函数是一等对象”意味着函数可以绑定给变量、放进容器、作为实参传入，也可以从另一个函数返回。区别首先在有没有调用括号：

```
def double(number: int) -> int:
    return number * 2


operation = double     # 函数对象
result = operation(5)  # 调用函数，结果是 10
```

传入函数：不先声明函数式接口

```
from collections.abc import Callable


def apply(value: int, operation: Callable[[int], int]) -> int:
    return operation(value)


apply(-4, abs)    # 4
apply(5, double)  # 10
```

`Callable[[int], int]` 表示“可以用一个 `int` 调用，并预期返回 `int` 的对象”。它可以先类比为 Java 的函数式接口类型，但 Python 不要求为每种函数签名单独声明接口，而且这个类型提示仍然不会自动在运行时校验。

`sorted(students, key=score_of)` 是同一种高阶函数模式：调用者注入“怎样提取分数”，`sorted` 负责遍历和排序。函数的职责因此可以保持分离。

返回函数：闭包把行为和外部环境放在一起

```
from collections.abc import Callable


def make_multiplier(factor: int) -> Callable[[int], int]:
    def multiply(value: int) -> int:
```

返回函数：闭包把行为和外部环境放在一起

```
from collections.abc import Callable


def make_multiplier(factor: int) -> Callable[[int], int]:
    def multiply(value: int) -> int:
        return value * factor

    return multiply


triple = make_multiplier(3)
triple(10)  # 30
```

外层调用结束后， `multiply` 仍然能访问当时所在作用域中的 `factor` 。这种“函数与它所引用的外部环境”称为**闭包**。它适合生成小型策略或回调；当状态、生命周期和行为逐渐复杂时，类通常更容易维护。

闭包捕获的是名字所在的绑定环境，不是每次创建时自动复制当前值。下面三个函数最终都会读到循环结束后的 `factor=4` ：

```
def make_operations():
    operations = []
    for factor in (2, 3, 4):
        operations.append(lambda value: value * factor)
    return operations


[operation(10) for operation in make_operations()]  # [40, 40, 40]
```

需要固定每轮的值时，优先调用 `make_multiplier(factor)` 这样的工厂函数。也可以利用默认值在函数创建时求值的规则写 `lambda value, factor=factor: value * factor` ，但工厂函数通常更清楚。

`lambda` 只适合一条短规则

`lambda` 创建匿名函数，函数体只能是一个表达式：

ME × README × +
olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html
期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

`lambda` 只适合一条短规则

`lambda` 创建匿名函数，函数体只能是一个表达式：

```
sorted(words, key=lambda word: (len(word), word))
```

它适合只用一次、含义清楚的转换或排序键。需要多步计算、异常处理、注释、复用或独立测试时，应改用有名字的 `def` 。

列表推导式与 `lambda` 不是同一种语法：

```
[student.name for student in students]  # 推导式负责遍历并创建列表
lambda student: student.name            # 匿名函数一次只处理一个实参
```

只有把后者交给 `map` 、 `sorted` 等负责遍历的函数，它才会依次作用于一批数据。

7.6 `*args` ： 收集或拆开位置实参

星号出现在定义侧和调用侧时，方向正好相反：

出现位置	含义
`def total(*values)`	收集：把剩余位置实参装进元组 `values`
`total(*numbers)`	拆开：把可迭代对象中的元素作为多个位置实参传入

定义侧：收集为元组

```
def total(*values: float) -> float:
    print(type(values), values)
    return sum(values)


total()                  # values == ()
total(10.0, 20.0, 30.0) # values == (10.0, 20.0, 30.0)
```

`args` 只是社区惯用名，真正起作用的是 `*` 。有明确业务含义时， `values` 、 `paths` 、 `tags` 通常比 `args` 更容易读懂。

s/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

固定位置参数可以放在 `*args` 前面；`*args` 后面的参数自动成为仅关键字参数：

```
def format_tags(prefix: str, *tags: str, separator: str = ", ") -> str:
    return f"{prefix}: {separator.join(tags)}"


format_tags("课程", "Python", "函数")
format_tags("课程", "Python", "函数", separator=" | ")
```

调用侧：拆开可迭代对象

```
numbers = [10.0, 20.0, 30.0]
total(*numbers)       # 等价于 total(10.0, 20.0, 30.0)
total(5.0, *numbers)  # 等价于 total(5.0, 10.0, 20.0, 30.0)
```

这与 Java varargs 有相似用途，但 Python 的 `*` 还可以在调用任意函数时拆开列表、元组、`range` 或生成器。即使语法允许，也不要用 `*args` 隐藏数量固定、含义不同的业务参数。

7.7 `**kwargs`：收集或拆开关键字实参

双星号同样需要区分定义侧和调用侧：

出现位置	含义
`def create_profile(**attributes)`	收集：把额外关键字实参装进字典
`create_profile(**options)`	拆开：把映射的键和值作为参数名和实参值

定义侧：收集为字典

```
def create_profile(name: str, **attributes: object) -> dict[str, object]:
    return {"name": name, **attributes}


profile = create_profile("Alice", age=28, city="Shanghai")
# attributes == {"age": 28, "city": "Shanghai"}
```

kwargs 也只是惯用名，使用 attributes、metadata 或 options 通常能更准确地表达这些

`kwargs` 也只是惯用名。使用 `attributes`、`metadata` 或 `options`，通常能更准确地表达这些额外参数的用途。

调用侧：拆开映射

```
options = {"age": 30, "city": "Beijing"}
create_profile("Bob", **options)
```

作为关键字参数展开时，映射的键必须是字符串。展开后仍然遵守普通绑定规则；同一参数收到两个值会抛出 `TypeError`：

```
options = {"name": "Bob"}
create_profile("Alice", **options)  # TypeError: name 得到了两个值
```

`*` 和 `**` 可以同时用于转发数据：

```
def summarize(*values: float, **metadata: object) -> dict[str, object]:
    return {
        "values": values,
        "total": sum(values),
        "metadata": metadata,
    }


numbers = [10.0, 20.0, 30.0]
options = {"author": "Alice", "unit": "分"}
summarize(*numbers, **options)
```

`**kwargs` 是开放扩展点，不是默认接口设计

`**kwargs` 适合装饰器转发、轻量适配层和真正开放的扩展点。稳定业务接口如果全靠它接收配置，拼写错误可能被藏进字典，IDE 也很难提供准确补全：

```
def request(url: str, **options: object): ...
request("https://example.com", tiemout=10)  # timeout 拼错，函数仍可能先接收下来
```

参数集合稳定时，明确列出仅关键字参数更可靠：

```
def request(
    url: str,
    *,
    timeout: float = 3.0,
    verify_ssl: bool = True,
) -> str:
    ...
```

配置字段很多、需要复用并且具有明确结构时，可以在后续章节使用数据类；如果必须保留字典形态，可以用 `TypedDict` 描述结构。两者都比无边界地扩张 `**kwargs` 更能表达契约。

Java 工程师最容易误判的地方

容易带过来的判断	实际行为	推荐做法
同名函数会像 Java 一样重载	后一次 `def` 重新绑定同一个名字	默认参数、明确命名，必要时使用专门分派机制
类型提示会阻止错误实参	默认只供静态分析，运行时仍会调用	使用类型检查器；业务边界显式校验
默认列表每次调用都会新建	默认对象在执行 `def` 时只创建一次	用 `None` 或专用哨兵，在函数体内创建
给形参赋新值会替换调用者变量	只重新绑定当前调用的局部名字	明确区分重新绑定和修改共享对象
`key` 像 Java `Comparator` 一样接收两个值	`key` 每次接收一个元素并返回比较键	写提取函数，例如 `key=lambda user: user.age`
闭包自动保存每轮循环值	闭包稍后按绑定环境读取名字	使用工厂函数创建闭包，或显式固定默认值
`*args` / `**kwargs` 越多越灵活越好	契约变模糊，错误更晚暴露	稳定接口优先明确参数和仅关键字参数

设计一个函数接口时，可以快速检查四件事：哪些参数必须提供，哪些值容易传反，函数是否修改传入的可变对象，扩展点是否真的需要接收任意参数。灵活语法的目标是让契约更清楚，而不是让所有调用都能“先传进来再说”。

小练习

练习 1：安全的默认值

实现 `add_tag(tag, tags=None)`。连续调用 `add_tag("Python")` 和 `add_tag("Java")`，确认得到两个互不影响的列表；再传入已有列表，观察原列表是否被修改。

练习 2：清晰的调用边界

实现：

```
def create_user(username, /, *, admin=False, active=True):
    ...
```

分别尝试合法调用与三种错误调用：使用 `username=`；把两个布尔值按位置传递；把 `active` 拼错。记录每次 `TypeError` 是在进入函数体前还是进入后发生。

练习 3：成绩报告

实现 `score_report(*scores, pass_line=60, **metadata)`，返回总分、平均分、是否全部及格、是否有人满分和附加信息。特别决定零个分数时平均分和“全部及格”应该是什么，不要直接让 `all([]) == True` 替业务做决定。

练习 4：URL 构建器

实现 `build_url(base, *paths, secure=True, **query)`：连接任意数量的路径，并使用标准库 `urllib.parse.urlencode` 编码查询参数。测试空路径、包含空格的值，以及同名查询参数需要多个值时应使用的数据结构。

运行本章示例

项目当前使用 Python 3.14；本章语法需要 Python 3.10+，并且只使用标准库，不需要安装第三方依赖。请在项目根目录运行：

```
python3 java_to_python/chapters/07_functions/example.py
```

ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-b84ee3bea292d7c88b1d03000f.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

练习 4：URL 构造器

实现 `build_url(base, *paths, secure=True, **query)`：连接任意数量的路径，并使用标准库 `urllib.parse.urlencode` 编码查询参数。测试空路径、包含空格的值，以及同名查询参数需要多个值时应使用的数据结构。

运行本章示例

项目当前使用 Python 3.14；本章语法需要 Python 3.10+，并且只使用标准库，不需要安装第三方依赖。请在项目根目录运行：

```
python3 java_to_python/chapters/07_functions/example.py
```

输出按 7.2～7.7 和 README 练习分段。运行时重点观察：

- 可变默认列表的两次返回是否指向同一个对象；
- `/`、`*` 如何限制 `price` 和 `connect` 的调用方式；
- 函数放进变量、字典、参数和返回值后何时真正执行；
- `*args` 在函数内是元组，`**kwargs` 在函数内是字典；
- 错误参数名、错误位置调用如何在函数体执行前抛出 `TypeError`。

若本机同时安装了 JDK，可以通过统一运行器对照 Python 与 Java 版本：

```
python3 java_to_python/run_all.py --chapter 7
```

统一运行器会把 Java 编译结果放进临时目录，不会在源码目录留下 `.class` 文件。想只运行 Python 时可追加 `--python-only`。

本章小结

- 调用函数时，Python 先计算实参并按签名完成绑定，成功后才执行函数体；
- 默认值在执行 `def` 时求值一次，可变默认对象会被多次调用共享；
- `/`、`*`、明确参数名和默认值共同构成函数的调用契约；
- 函数是对象，因此能赋值、传入、返回并形成闭包；
- 定义侧的 `*`、`**` 负责收集，调用侧的 `*`、`**` 负责拆开；
- `*args` 和 `**kwargs` 提供扩展能力，但稳定业务接口仍应优先表达明确契约。

```python
"""第 7 章：函数参数、内置函数、一等函数、*args 与 **kwargs。

这个文件故意把每个知识点拆成独立示例。直接运行后，可以按照输出中的章节标题，
把调用方式、函数内部收到的值和最终结果对照起来看。
"""

from collections.abc import Callable
from urllib.parse import quote, urlencode


def show_section(title: str) -> None:
    """让不同知识点的运行结果有清晰边界。"""
    print(f"\n{'=' * 12} {title} {'=' * 12}")


# ---------------------------------------------------------------------------
# 7.2 位置参数与默认参数
# ---------------------------------------------------------------------------
def calculate_total(
    prices: list[float],
    discount: float = 0.0,
    *,
    shipping: float = 0.0,
) -> float:
    """计算订单总价。

    prices 和 discount 可以按位置传入；discount 省略时使用 0.0。
    单独的 * 表示它后面的 shipping 只能写成 ``shipping=...``，这样调用处不会
    把“折扣”和“运费”这两个同为 float 的参数弄反。
    """
    subtotal: float | Literal[0] = sum(prices)
    discounted: float = subtotal * (1 - discount)
    return discounted + shipping


def unsafe_collect(
    item: str,
```

```
def unsafe_collect(
    item: str,
    target: list[str] = [],  # noqa: B006  # 教学反例：故意展示可变默认值
) -> list[str]:
    """错误示范：默认列表只创建一次，之后每次省略 target 都会共享它。"""
    target.append(item)
    return target


def collect(item: str, target: list[str] | None = None) -> list[str]:
    """正确示范：省略 target 时，每次调用都创建一个全新的列表。

    注意：调用者如果明确传入列表，本函数仍会修改那个列表。
    这往往正是调用者想要的；
    如果不希望修改外部列表，应先写 ``actual_target = list(target)`` 创建副本。
    """
    actual_target: list[str] = [] if target is None else target
    actual_target.append(item)
    return actual_target


def add_tag(tag: str, tags: list[str] | None = None) -> list[str]:
    """省略 tags 时每次创建新列表；明确传入时按约定修改原列表。"""
    actual_tags: list[str] = [] if tags is None else tags
    actual_tags.append(tag)
    return actual_tags


# ---------------------------------------------------------------------------
# 7.3 关键字参数、仅位置参数和仅关键字参数
# ---------------------------------------------------------------------------
def price(
    amount: float,
    /,
    tax_rate: float = 0.13,
    *,
    discount: float = 0.0
```

```
64  # ------------------------------------------------------------------------
65  # 7.3 关键字参数、仅位置参数和仅关键字参数
66  # ------------------------------------------------------------------------
67  def price(
68      amount: float,
69      /,
70      tax_rate: float = 0.13,
71      *,
72      discount: float = 0.0,
73  ) -> float:
74      """展示三类参数。
75
76      amount 在 / 前面，只能按位置传递。
77      tax_rate 在 / 和 * 之间，可以按位置传递，也可以按名字传递。
78      discount 在 * 后面，只能按名字传递。
79
80      以下调用会报 TypeError，因此没有放进 main 执行：
81          price(amount=100.0)          # amount 不能用关键字传递
82          price(100.0, 0.13, 0.2)     # discount 不能作为第三个位置参数
83      """
84      discounted: float = amount * (1 - discount)
85      return round(discounted * (1 + tax_rate), 2)
86
87
88  def connect(
89      host: str,
90      port: int = 5432,
91      *,
92      use_ssl: bool = True,
93      timeout: float = 3.0,
94  ) -> str:
95      """布尔开关和多个数字参数使用关键字后，调用代码更容易读懂。"""
96      protocol: Literal['加密连接'] | Literal['普通连接'] = "加密连接" if use_ssl else "普通连接"
97      return f"{protocol} {host}:{port}, 超时 {timeout} 秒"
98
99
100 def create_user(
```

```python
def create_user(
    username: str,
    /,
    *,
    admin: bool = False,
    active: bool = True,
) -> dict[str, object]:
    """用 / 和 * 让用户名仅限位置传递、开关仅限关键字传递。"""
    return {"username": username, "admin": admin, "active": active}


# ------------------------------------------------------------------------
# 7.5 函数也是对象
# ------------------------------------------------------------------------
def double(number: int) -> int:
    return number * 2


def square(number: int) -> int:
    return number**2


def apply(value: int, operation: Callable[[int], int]) -> int:
    """operation 是一个参数：它必须接收 int，并返回 int。"""
    return operation(value)


def score_of(student: tuple[str, int]) -> int:
    """作为 sorted 的 key 函数：每次接收一名学生，返回用于比较的分数。"""
    return student[1]


def make_multiplier(factor: int) -> Callable[[int], int]:
    """返回一个新函数；新函数会记住本次调用传入的 factor。"""

    def multiply(value: int) -> int:
```

```python
def make_multiplier(factor: int) -> Callable[[int], int]:
    """返回一个新函数；新函数会记住本次调用传入的 factor。"""

    def multiply(value: int) -> int:
        return value * factor

    return multiply


# ---------------------------------------------------------------------------
# 7.6 任意数量的位置参数：*args
# ---------------------------------------------------------------------------
def total(*values: float) -> float:
    """values 在函数内部一定是元组；没有参数时，它就是空元组。"""
    return sum(values)


def describe_args(*values: object) -> str:
    """把 *args 的真实类型和值显示出来，便于观察。"""
    return f"类型={type(values).__name__}, 值={values}"


def format_tags(prefix: str, *tags: str, separator: str = ", ") -> str:
    """固定参数、*args 和仅关键字参数可以组合使用。

    prefix 接收第一个位置参数，剩余位置参数进入 tags；因为 separator 位于 *tags
    后面，所以它是仅关键字参数。
    """
    return f"{prefix}: {separator.join(tags)}"


# ---------------------------------------------------------------------------
# 7.7 任意数量的关键字参数：**kwargs
# ---------------------------------------------------------------------------
def create_profile(name: str, **attributes: object) -> dict[str, object]:
    """attributes 在函数内部是字典，保存调用者额外传入的命名参数。"""
```

```python
# ------------------------------------------------------------------------------
# 7.7 任意数量的关键字参数：**kwargs
# ------------------------------------------------------------------------------
def create_profile(name: str, **attributes: object) -> dict[str, object]:
    """attributes 在函数内部是字典，保存调用者额外传入的命名参数。"""
    return {"name": name, **attributes}


def summarize(*values: float, **metadata: object) -> dict[str, object]:
    """同时接收任意位置参数和任意关键字参数。"""
    return {
        "values": values,
        "count": len(values),
        "total": sum(values),
        "metadata": metadata,
    }


def score_report(
    *scores: int,
    pass_line: int = 60,
    **metadata: object,
) -> dict[str, object]:
    """汇总任意数量的成绩；没有成绩时用 None 表示没有平均值。"""
    if not 0 <= pass_line <= 100:
        raise ValueError("及格线必须在 0 到 100 之间")
    if any(not 0 <= score <= 100 for score in scores):
        raise ValueError("成绩必须在 0 到 100 之间")

    total_score: int = sum(scores)
    average_score: float | None = total_score / len(scores) if scores else None
    return {
        "count": len(scores),
        "total": total_score,
        "average": average_score,
        # all([]) 虽然为 True，但业务上“没有学生”不等于“全部及格”。
```

```
181  def score_report(
194      return {
195          "count": len(scores),
196          "total": total_score,
197          "average": average_score,
198          # all([]) 虽然为 True，但业务上“没有学生”不等于“全部及格”。
199          "all_passed": bool(scores) and all(score >= pass_line for score in scores),
200          "has_perfect_score": any(score == 100 for score in scores),
201          "metadata": metadata,
202      }
203
204
205  def build_url(
206      base: str,
207      *paths: str,
208      secure: bool = True,
209      **query: object,
210  ) -> str:
211      """连接 URL 路径，并用标准库正确编码查询参数。"""
212      _: str, separator: str, host_part: str = base.partition("://")
213      host: str = host_part if separator else base
214      scheme: Literal['https'] | Literal['http'] = "https" if secure else "http"
215      url: str = f"{scheme}://{host.strip('/')}"
216
217      # 每个 path 实参代表一个路径段；空段忽略，特殊字符进行百分号编码。
218      encoded_paths: list[str] = [quote(path.strip("/"), safe="") for path in paths if path.strip("/")]
219      if encoded_paths:
220          url: str = f"{url}/{'/'.join(encoded_paths)}"
221
222      # doseq=True 允许值使用列表/元组，表达同名查询参数出现多次。
223      encoded_query: str = urlencode(query, doseq=True)
224      return f"{url}?{encoded_query}" if encoded_query else url
225
226
227  def inspect_call(
228      identifier: int,
```

```python
226
227 def inspect_call(
228     identifier: int,
229     /,
230     category: str = "general",
231     *tags: str,
232     active: bool = True,
233     **extra: object,
234 ) -> dict[str, object]:
235     """集中展示参数定义的常见完整顺序。
236
237     1. identifier: 仅位置参数
238     2. category: 普通参数（位置或关键字均可）
239     3. tags: 收集剩余位置参数的元组
240     4. active: 仅关键字参数
241     5. extra: 收集剩余关键字参数的字典
242     """
243     return {
244         "identifier": identifier,
245         "category": category,
246         "tags": tags,
247         "active": active,
248         "extra": extra,
249     }
250
251
252 def request(
253     url: str,
254     *,
255     timeout: float = 3.0,
256     verify_ssl: bool = True,
257 ) -> str:
258     """稳定接口优先列出参数，让拼写错误和类型问题更容易被 IDE 发现。"""
259     return f"GET {url} (timeout={timeout}, verify_ssl={verify_ssl})"
260
261
262 def demonstrate_defaults() -> None:
```

```python
def demonstrate_defaults() -> None:
    show_section("7.2 位置参数与默认参数")

    # discount 被省略，使用默认值 0.0。
    print("使用默认折扣:", calculate_total([100.0, 50.0]))

    # 前两个参数按位置匹配；shipping 被 * 限制为只能按名字传递。
    print("位置参数 + 关键字参数:", calculate_total([100.0, 50.0], 0.1, shipping=8.0))

    # 错误版本的两次调用没有传 target，因此拿到的是同一个默认列表。
    unsafe_first: list[str] = unsafe_collect("A")
    unsafe_second: list[str] = unsafe_collect("B")
    print("可变默认值第一次结果也被改变:", unsafe_first)
    print("两次返回同一个列表:", unsafe_first is unsafe_second)

    # 正确版本会在函数体内分别创建列表。
    safe_first: list[str] = collect("A")
    safe_second: list[str] = collect("B")
    print("None 哨兵得到独立列表:", safe_first, safe_second)
    print("两次返回不同列表:", safe_first is not safe_second)

    # 明确传入列表时，collect 会按设计修改调用者的列表。
    shopping_cart: list[str] = ["苹果"]
    returned_cart: list[str] = collect("香蕉", shopping_cart)
    print("传入已有列表:", shopping_cart)
    print("返回的正是传入列表:", returned_cart is shopping_cart)


def demonstrate_keyword_arguments() -> None:
    show_section("7.3 关键字参数与 /、*")

    print("全部使用默认选项:", price(100.0))
    print("tax_rate 按位置传入:", price(100.0, 0.10, discount=0.20))
    print("tax_rate 按名字传入:", price(100.0, tax_rate=0.10, discount=0.20))

    # 如果只看到 connect("db.example.com", 5432, False, 10)，很难立刻知道 False
```

```
290  def demonstrate_keyword_arguments() -> None:
297      # 如果只看到 connect("db.example.com", 5432, False, 10)，很难立刻知道 False
298      # 和 10 的含义。仅关键字参数把含义直接写在调用处。
299      print(
300          "清晰的配置调用:",
301          connect("db.example.com", use_ssl=False, timeout=10.0),
302      )
303
304
305  def demonstrate_builtins() -> None:
306      show_section("7.4 常用内置函数")
307
308      # len 面向"有长度"协议工作，不局限于某一种集合。
309      print("len 字符串/列表/字典:", len("Python"), len([10, 20, 30]), len({"name": "Alice"}))
310
311      scores: list[int] = [88, 95, 72, 100]
312      print("sum/min/max:", sum(scores), min(scores), max(scores))
313
314      # sorted 返回新列表，不修改原数据；list.sort() 才会原地修改列表。
315      sorted_scores: list[int] = sorted(scores, reverse=True)
316      print("sorted 降序结果:", sorted_scores)
317      print("原列表保持不变:", scores)
318
319      words: list[str] = ["pear", "fig", "banana", "kiwi"]
320      print("按长度排序:", sorted(words, key=len))
321
322      # enumerate 同时产生序号和值，start=1 适合面向用户的编号。
323      for index, word in enumerate(words, start=1):
324          print(f"enumerate -> {index}. {word}")
325
326      names: list[str] = ["Alice", "Bob", "Cara"]
327      points: list[int] = [95, 88, 91]
328      # strict=True 能在两边长度不一致时抛出 ValueError，避免数据被静默截断。
329      print("zip 配对:", list(zip(names, points, strict=True)))
330
331      print("是否有人满分 any:", any(score == 100 for score in scores))
332      print("是否全部及格 all:", all(score >= 60 for score in scores))
```

```
305 def demonstrate_builtins() -> None:
331     print("是否有人满分 any:", any(score == 100 for score in scores))
332     print("是否全部及格 all:", all(score >= 60 for score in scores))
333     print("空序列 any/all:", any([]), all([]))
334
335
336 def demonstrate_function_objects() -> None:
337     show_section("7.5 函数也是对象")
338
339     # 赋值时没有括号：保存的是函数本身；operation(6) 才真正调用函数。
340     operation: Callable[..., int] = double
341     print("函数赋给变量:", operation(6))
342
343     # 函数可以像字符串或数字一样放进容器。
344     operations: dict[str, Callable[[int], int]] = {
345         "double": double,
346         "square": square,
347     }
348     for name, current_operation in operations.items():
349         print(f"函数放入字典 {name}:", current_operation(5))
350
351     # apply 不知道具体运算，只负责调用传入的函数。
352     # 这就是“把行为作为参数”。
353     print("函数作为参数:", apply(-4, abs), apply(5, square))
354
355     students: list[tuple[str, int]] = [("Alice", 88), ("Bob", 95), ("Cara", 91)]
356     print("函数作为排序规则:", sorted(students, key=score_of, reverse=True))
357
358     # make_multiplier 返回函数；triple 会记住 factor=3。
359     # 这种记忆外层变量的函数叫闭包。
360     triple: Callable[[int], int] = make_multiplier(3)
361     print("函数作为返回值:", triple(10))
362
363     # lambda 适合非常短的一次性表达式。
364     # 有分支、循环或异常处理时应改用 def。
365     print("简单 lambda:", apply(5, lambda number: number + 1))
```

```
336  def demonstrate_function_objects() -> None:
367      # 列表推导式自己负责遍历并创建列表；它不是 lambda。
368      student_names_by_comprehension: list[str] = [student[0] for student in students]
369      # 下面才出现 lambda：map 负责遍历，lambda 只描述“怎样处理一个 student”。
370      student_names_by_map: list[str] = list(  # noqa: C417  # 教学对照：故意展示 lambda + map
371          map(lambda student: student[0], students)
372      )
373      print("列表推导式取姓名:", student_names_by_comprehension)
374      print("lambda + map 取姓名:", student_names_by_map)
375
376
377  def demonstrate_args() -> None:
378      show_section("7.6 任意数量的位置参数 *args")
379
380      print("零个位置参数:", total())
381      print("多个位置参数:", total(10.0, 20.0, 30.0))
382      print("函数内部看到的 args:", describe_args("A", 10, True))
383
384      numbers: list[float] = [10.0, 20.0, 30.0]
385      # 调用侧的 * 做“拆包”：等价于 total(10.0, 20.0, 30.0)。
386      print("把列表拆成位置参数:", total(*numbers))
387      print("固定参数后继续拆包:", total(5.0, *numbers))
388
389      print("固定参数 + args:", format_tags("课程", "Python", "函数", "参数"))
390      print(
391          "args 后的仅关键字参数:",
392          format_tags("课程", "Python", "函数", separator=" | "),
393      )
394
395
396  def demonstrate_kwargs() -> None:
397      show_section("7.7 任意数量的关键字参数 **kwargs")
398
399      print("直接传关键字参数:", create_profile("Alice", age=28, city="Shanghai"))
400
401      profile_options: dict[str, object] = {"age": 30, "city": "Beijing", "verified": True}
```

```
396  def demonstrate_kwargs() -> None:
401      profile_options: dict[str, object] = {"age": 30, "city": "Beijing", "verified": True}
402      # 调用侧的 ** 做“拆包”：字典的键变成参数名，值变成对应的参数值。
403      print("把字典拆成关键字参数:", create_profile("Bob", **profile_options))
404
405      numbers: list[float] = [10.0, 20.0, 30.0]
406      metadata: dict[str, object] = {"author": "Cara", "unit": "分"}
407      print("同时使用 * 和 **:", summarize(*numbers, **metadata))
408
409      print(
410          "完整参数顺序:",
411          inspect_call(
412              1001,
413              "course",
414              "Python",
415              "beginner",
416              active=True,
417              owner="Alice",
418              lessons=10,
419          ),
420      )
421
422      # 对稳定接口，明确参数通常优于把所有配置都塞进 **kwargs。
423      print("明确的业务参数:", request("https://example.com", timeout=10.0, verify_ssl=False))
424
425
426  def demonstrate_readme_exercises() -> None:
427      """运行 README 中的四组函数设计练习。"""
428      show_section("README 小练习")
429
430      print("练习 1: 安全的默认值")
431      python_tags: list[str] = add_tag("Python")
432      java_tags: list[str] = add_tag("Java")
433      print("  两次省略参数:", python_tags, java_tags)
434      print("  两个结果互不影响:", python_tags is not java_tags)
435      existing_tags: list[str] = ["backend"]
436      returned_tags: list[str] = add_tag("api", existing_tags)
```

```
426 def demonstrate_readme_exercises() -> None:
435     existing_tags: list[str] = ["backend"]
436     returned_tags: list[str] = add_tag("api", existing_tags)
437     print("  明确传入的原列表被修改:", existing_tags)
438     print("  返回值就是原列表:", returned_tags is existing_tags)
439
440     print("练习 2: 清晰的调用边界")
441     print("  合法调用:", create_user("Alice", admin=True, active=False))
442     try:
443         create_user(username="Bob")  # type: ignore[call-arg]
444     except TypeError as error:
445         print("  username= 非法:", error)
446     try:
447         create_user("Bob", True, False)  # type: ignore[call-arg]
448     except TypeError as error:
449         print("  布尔值按位置传递非法:", error)
450     try:
451         create_user("Bob", actve=True)  # type: ignore[call-arg]
452     except TypeError as error:
453         print("  拼错 active 非法:", error)
454
455     print("练习 3: 成绩报告")
456     print(
457         "  有成绩:",
458         score_report(88, 60, 100, pass_line=60, course="Python", teacher="Alice"),
459     )
460     print("  零个成绩:", score_report(course="Python"))
461
462     print("练习 4: URL 构建器")
463     print("  空路径:", build_url("api.example.com"))
464     print(
465         "  带路径和空格查询值:",
466         build_url("api.example.com", "users", "search", keyword="python 文件"),
467     )
468     print(
469         "  同名查询参数使用列表:",
```

```
426  def demonstrate_readme_exercises() -> None:
466          build_url("api.example.com", "users", "search", keyword="python 文件"),
467      )
468      print(
469          "  同名查询参数使用列表:",
470          build_url("api.example.com", "courses", tag=["python", "java"]),
471      )
472
473
474  def main() -> None:
475      demonstrate_defaults()
476      demonstrate_keyword_arguments()
477      demonstrate_builtins()
478      demonstrate_function_objects()
479      demonstrate_args()
480      demonstrate_kwargs()
481      demonstrate_readme_exercises()
482
483
484  if __name__ == "__main__":
485      main()
486
```